

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 9_COD

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers and a sum B, find all unique combinations in the array where the sum is equal to B. The same number may be chosen from the array any number of times to make B.

1. All numbers will be positive integers.
2. Elements in a combination ($a_1, a_2, \dots, a_k$) must be in non-descending order. (ie, $a_1 \leq a_2 \leq \dots \leq a_k$).
3. The combinations themselves must be sorted in ascending order.

Examples:

Input:

$\text{arr}[] = \{7, 2, 6, 5\}$

$B = 16$

Output:

(2 2 2 2 2 2 2 2)

(2 2 2 2 2 6)

(2 2 2 5 5)

(2 2 5 7)

(2 2 6 6)

(2 7 7)

(5 5 6)

Input:

$\text{arr}[] = \{6, 5, 7, 1, 8, 2, 9, 9, 7, 7, 9\}$

$B = 6$

Output:

(1 1 1 1 1 1)

(1 1 1 1 2)

(1 1 2 2)

(1 5)

(2 2 2)

(6)

Input Format

The first line of input consists of space-separated positive integers representing the elements of the array.

The second line consists of a single positive integer representing the target sum B .

Output Format

The output prints "Combinations that sum to [B] are:" followed by the combinations in parentheses, each combination separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7 2 6 5

16

Output: Combinations that sum to 16 are:

(2 2 2 2 2 2 2) (2 2 2 2 2 6) (2 2 2 5 5) (2 2 5 7) (2 2 6 6) (2 7 7) (5 5 6)

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int arr[20], res[50], n, target;
```

```
void sort() {  
    for (int i = 0; i < n-1; i++)  
        for (int j = i+1; j < n; j++)  
            if (arr[i] > arr[j]) {  
                int t = arr[i];  
                arr[i] = arr[j];  
                arr[j] = t;  
            }  
}
```

```
void find(int start, int sum, int idx) {  
    if (sum == target) {  
        printf(" ");  
        for (int i = 0; i < idx; i++)  
            printf("%d ", res[i]);  
        printf(" ");  
        return;  
    }
```

```
    for (int i = start; i < n; i++) {  
        if (sum + arr[i] <= target) {
```

```

        res[idx] = arr[i];
        find(i, sum + arr[i], idx + 1);
    }
}

int main() {
    n = 0;
    while (scanf("%d", &arr[n]) == 1) n++;

    target = arr[n-1];
    n--;

    sort();

    printf("Combinations that sum to %d are:\n", target);

    find(0, 0, 0);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Imagine you're helping a friend plan gift packages with a total budget of N. Your task is to find all unique ways to spend the entire budget on different combinations of gift prices, ensuring no combination is repeated by rearranging the prices.

Note:

By unique, it means that no matter that no other composition can be expressed as a permutation of the generated composition. For eg. [1,2,1] and [1, 1, 2] are not unique. You need to print all combinations in non-decreasing order, for eg. [1, 2, 1] or [1,1,2] will be printed as [1,1,2]. The order of printing all the combinations should be in lexicographical order.

Use backtracking to solve the given problem.

Input Format

The input consists of a single integer, n, which represents the number to be broken down.

Output Format

The output consists of a list of lists, where each inner list represents a possible breakdown of the given number.

Each breakdown is represented as a list of integers that add up to the given number.

The breakdowns are printed in lexicographic order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Output: [[1]]

Answer

```
// You are using GCC
#include <iostream>
using namespace std;
```

```
int n, res[50];
```

```
void solve(int start, int sum, int idx, bool &first) {
    if (sum == n) {
        if (!first) cout << ", ";
        cout << "[";
        for (int i = 0; i < idx; i++) {
            cout << res[i];
            if (i != idx - 1) cout << ", ";
        }
        cout << "]";
        first = false;
        return;
    }
```

```

for (int i = start; i <= n; i++) {
    if (sum + i <= n) {
        res[idx] = i;
        solve(i, sum + i, idx + 1, first);
    }
}
}
}

```

```

int main() {
    cin >> n;

    bool first = true;
    cout << "[";

    solve(1, 0, 0, first);

    cout << "]";

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Rohan is designing a security system that generates all possible rearrangements of a given access code made up of distinct characters. He inputs a string representing the access code into the system. The system must display every possible permutation of the characters without repetition.

Write a program to ensure that all permutations of the given string are printed in separate lines.

Input Format

The input consists of a single line containing a string S, consisting of lowercase or upper-case alphabets.

Output Format

The output prints each permutation of the string on a new line.

The permutations should be printed in the order generated by swapping characters starting from the first position

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: ABC

Output: ABC

ACB

BAC

BCA

CBA

CAB

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void swap(char *a, char *b) {  
    char t = *a;  
    *a = *b;  
    *b = t;  
}
```

```
void permute(char str[], int l, int r) {  
    if (l == r) {  
        printf("%s\n", str);  
        return;  
    }  
}
```

```
    for (int i = l; i <= r; i++) {  
        swap(&str[l], &str[i]);  
        permute(str, l + 1, r);  
        swap(&str[l], &str[i]);  
    }  
}
```

```
int main() {  
    char str[10];  
    scanf("%s", str);  
  
    int n = strlen(str);  
    permute(str, 0, n - 1);  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Given a number K and string str of digits denoting a positive integer, build the largest number possible by performing swap operations on the digits of str at most K times.

Input Format

The first line of input consists of a positive integer K.

The second line of input consists of a string of digits of positive integers.

Output Format

The output displays the largest number possible by performing swap operations on the digits of str at most K times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

254

Output: 524

Answer

// You are using GCC

```
#include <stdio.h>
#include <string.h>
```

```
char maxStr[50];
```

```
void swap(char *a, char *b) {
    char t = *a;
    *a = *b;
    *b = t;
}
```

```
void solve(char str[], int k, int n) {
    if (k == 0) return;
```

```
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
```

```
            if (str[j] > str[i]) {
                swap(&str[i], &str[j]);
```

```
            if (strcmp(str, maxStr) > 0)
                strcpy(maxStr, str);
```

```
            solve(str, k - 1, n);
```

```
            swap(&str[i], &str[j]);
```

```
        }
    }
}
```

```
int main() {
    int k;
    char str[50];
```

```
    scanf("%d", &k);
    scanf("%s", str);
```

```
    strcpy(maxStr, str);
```

```
    int n = strlen(str);
```

```
solve(str, k, n);  
printf("%s", maxStr);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

5. Problem Statement

Mythili has a list of item prices and a customer's gift card balance. She needs to find a combination of two or more items whose total price exactly matches the gift card balance. If such a combination exists, she should print the item prices in the order they appear. If multiple valid combinations exist, she must print the first one found. If no valid combination is possible, print "No combination of items"

Write a function that takes the list of item prices, the number of items, and the gift card balance as input, and prints the result based on the above rules.

Input Format

The first line of input consists of an integer, representing the number of items in the store.

The second line consists of an array of integers separated by a space, representing the prices of the items.

The last line consists of an integer representing the customer's gift card balance.

Output Format

If there exists a combination of items whose prices sum up to the gift card balance, then print this combination.

If no such combination exists, print "No combination of items".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
50

Output: 20 30

Answer

```
#include <stdio.h>
```

```
int found = 0;
```

```
void solve(int arr[], int n, int idx, int sum, int target, int res[], int r) {  
    if (sum == target && r >= 2 && !found) {  
        for (int i = 0; i < r; i++) {  
            printf("%d", res[i]);  
            if (i != r - 1) printf(" ");  
        }  
        found = 1;  
        return;  
    }  
}
```

```
if (idx == n || sum > target || found)  
    return;
```

```
res[r] = arr[idx];  
solve(arr, n, idx + 1, sum + arr[idx], target, res, r + 1);
```

```
    solve(arr, n, idx + 1, sum, target, res, r);  
}
```

```
int main() {  
    int n, target;  
    scanf("%d", &n);
```

```
    int arr[10];  
    for (int i = 0; i < n; i++)  
        scanf("%d", &arr[i]);
```

```
    scanf("%d", &target);
```

```
int res[10];  
solve(arr, n, 0, 0, target, res, 0);  
  
if (!found)  
    printf("No combination of items");  
  
return 0;  
}
```

Status : Correct

Marks : 10/10